AI Raw Response

```
content = {
  "tree": """.
■■■ .gitattributes
■■■ .gitignore
■■■ .github
■   ■■■ workflows
■       ■■■ ci.yml
■■■ README.md
■■■ docker-compose.yml
■■■ backend
■   ■■■ .env.example
■   ■■■ Dockerfile
■   ■■■ requirements.txt
■   ■■■ app
■   ■   ■■■ __init__.py
■   ■   ■■■ config.py
■   ■   ■■■ logging_config.py
■   ■   ■■■ main.py
■   ■   ■■■ utils
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ file_utils.py
■   ■   ■■■ models
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ db_models.py
■   ■   ■   ■■■ schemas.py
■   ■   ■■■ db
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ session.py
■   ■   ■   ■■■ vectorstore.py
■   ■   ■■■ processing
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ parser.py
■   ■   ■   ■■■ chunker.py
■   ■   ■   ■■■ embedding.py
■   ■   ■■■ retrieval
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ retriever.py
■   ■   ■■■ services
■   ■       ■■■ __init__.py
■   ■       ■■■ exam_generator.py
■   ■       ■■■ indexer.py
■   ■■■ tests
■       ■■■ test_health.py
■■■ frontend
    ■■■ app.json
    ■■■ babel.config.js
    ■■■ metro.config.js
    ■■■ package.json
    ■■■ tsconfig.json
    ■■■ app
        ■■■ App.tsx
        ■■■ api
        ■   ■■■ client.ts
        ■■■ components
        ■   ■■■ BookItem.tsx
        ■   ■■■ TopicList.tsx
        ■■■ hooks
        ■   ■■■ useBooks.ts
        ■■■ screens
            ■■■ BookScreen.tsx
```

```
            ■■■ ExamConfigScreen.tsx
            ■■■ ExamViewScreen.tsx
            ■■■ LibraryScreen.tsx
            ■■■ UploadScreen.tsx
""",
  ".gitattributes": """* text=auto eol=lf
*.bat text eol=crlf
*.sh text eol=lf
*.md text eol=lf
*.yml text eol=lf
*.yaml text eol=lf
*.json text eol=lf
*.ts text eol=lf
*.tsx text eol=lf
*.py text eol=lf
""",
  ".gitignore": """# General
.DS_Store
*.log
*.tmp
*.swp
*.swo
.env
.env.*
venv/
.venv/

# Python
__pycache__/
*.py[cod]
*.egg-info/
dist/
build/

# Node/React Native/Expo
node_modules/
.expo/
.expo-shared/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
package-lock.json

# IDE
.vscode/
.idea/

# Data
data/
uploads/
indexes/
""",
  ".github/workflows/ci.yml": """name: CI

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
```

```
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install deps
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Lint import
        run: python -m pyflakes app || true
      - name: Test
        run: pytest -q || true

  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: 'npm'
          cache-dependency-path: frontend/package.json
      - name: Install deps
        run: npm ci
      - name: Build
        run: npm run lint && npm run tsc --noEmit
""",
  "README.md": """# LLM Exam Builder: Libros a Exámenes Personalizados

Sistema completo para:
- Subir libros (PDF) y procesarlos.
- Tokenizar preservando estructura (temas, subtemas, ejercicios).
- Indexar en FAISS con embeddings por tema y dificultad.
- Explorar temas indexados en una interfaz navegable.
- Generar exámenes personalizados con x preguntas por temas seleccionados.

Tecnologías:
- Backend: Python (FastAPI), LangChain, FAISS, PyMuPDF, sentence-transformers, SQLModel/SQ
- Frontend: React Native (Expo) con soporte Web.
- Infra: Docker, GitHub Actions.

Estructura
- backend: API y procesado.
- frontend: App Expo (nativa y web).

Requisitos
- Python 3.11+
- Node 18+ y npm
- Docker (opcional)
- En producción puedes configurar OpenAI para generación más potente. Por defecto, usa emb
locales y generación básica.
```

Variables de Entorno (backend/.env)
- APP_ENV=dev
- DATA_DIR=./data
- UPLOAD_DIR=./uploads
- INDEX_DIR=./indexes
- DATABASE_URL=sqlite:///./data/app.db
- EMBEDDINGS_PROVIDER=huggingface # huggingface | openai
- EMBEDDINGS_MODEL=sentence-transformers/all-MiniLM-L6-v2
- OPENAI_API_KEY= # opcional
- LLM_MODEL=openai/gpt-4o-mini # o llama/open-source; se ignora si no hay API key
- MAX_CHUNK_TOKENS=600
- CHUNK_OVERLAP_TOKENS=80

Arranque en local (sin Docker)
1) Backend
   - cp backend/.env.example backend/.env
   - python -m venv .venv && source .venv/bin/activate
   - pip install -r backend/requirements.txt
   - uvicorn app.main:app --reload --host 0.0.0.0 --port 8000 --app-dir backend

   Endpoints:
   - POST /api/books/upload (multipart/form-data: file, title)
   - GET /api/books
   - GET /api/books/{book_id}/topics
   - POST /api/exams/generate

2) Frontend (Expo)
   - cd frontend
   - npm install
   - npm start
   - Para web: presiona 'w'. Para móvil: usa Expo Go.

Arranque con Docker
- docker-compose up --build
- Backend en http://localhost:8000
- Frontend web en http://localhost:19006 (Expo web)

Despliegue
- Construye imágenes:
  - docker build -t yourorg/exam-backend:latest ./backend
  - docker build -t yourorg/exam-frontend:latest ./frontend
- Despliega en tu orquestador (ECS, K8s, Render, Fly.io).
- Asegura volúmenes persistentes para data/, uploads/, indexes/.

Flujo de uso
1) Sube un PDF desde la UI (Upload).
2) El backend procesa en background: extrae texto (PyMuPDF), detecta temas y ejercicios heurísticamente, crea chunks token-aware, genera embeddings y guarda FAISS por libro.
3) Explora temas en la app (scroll).
4) Selecciona temas y genera examen con x preguntas. Si hay OpenAI, el LLM redacta; si no, genera una versión básica basada en el contenido y ejercicios detectados.

Extensibilidad
- Nuevos formatos (EPUB, DOCX): implementa parsers en backend/app/processing/parser.py sig
interfaz BaseParser.
- Nuevos proveedores de embeddings/LLM: agrega ramas en embedding.py y exam_generator.py.
- Nuevos tipos de evaluación: añade generadores en services/exam_generator.py.

Buenas prácticas implementadas
- Estructura modular.
- Configuración por entorno.

```
- Logging estructurado.
- Indexación incremental sin downtime (background tasks y rutas idempotentes).
- Vectorstores por libro (carga perezosa).

Licencia
- MIT (ajustar según necesidades).
""",
  "docker-compose.yml": """version: '3.9'

services:
  backend:
    build: ./backend
    env_file:
      - ./backend/.env
    ports:
      - "8000:8000"
    volumes:
      - ./backend/data:/app/data
      - ./backend/uploads:/app/uploads
      - ./backend/indexes:/app/indexes
    restart: unless-stopped

  frontend:
    build: ./frontend
    environment:
      - EXPO_DEVTOOLS_LISTEN_ADDRESS=0.0.0.0
      - API_BASE_URL=http://localhost:8000
    ports:
      - "19000:19000"
      - "19001:19001"
      - "19002:19002"
      - "19006:19006"
    depends_on:
      - backend
    stdin_open: true
    tty: true
""",
  "backend/.env.example": """APP_ENV=dev
DATA_DIR=./data
UPLOAD_DIR=./uploads
INDEX_DIR=./indexes
DATABASE_URL=sqlite:///./data/app.db
EMBEDDINGS_PROVIDER=huggingface
EMBEDDINGS_MODEL=sentence-transformers/all-MiniLM-L6-v2
OPENAI_API_KEY=
LLM_MODEL=gpt-4o-mini
MAX_CHUNK_TOKENS=600
CHUNK_OVERLAP_TOKENS=80
LOG_LEVEL=INFO
""",
  "backend/requirements.txt": """fastapi==0.115.0
uvicorn[standard]==0.30.6
python-multipart==0.0.9
pydantic==2.9.2
sqlmodel==0.0.21
SQLAlchemy==2.0.35
faiss-cpu==1.8.0.post1
langchain==0.3.2
langchain-community==0.3.1
sentence-transformers==3.1.1
tiktoken==0.7.0
```

```
PyMuPDF==1.24.10
aiofiles==24.1.0
numpy==2.1.2
requests==2.32.3
tenacity==9.0.0
pyyaml==6.0.2
pyflakes==3.2.0
pytest==8.3.3
""",
  "backend/Dockerfile": """FROM python:3.11-slim

# System deps
RUN apt-get update && apt-get install -y build-essential poppler-utils git && rm -rf
/var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Create dirs
RUN mkdir -p /app/data /app/uploads /app/indexes

COPY app /app/app

ENV PYTHONUNBUFFERED=1
ENV HOST=0.0.0.0
ENV PORT=8000

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
""",
  "backend/app/__init__.py": """__all__ = []""",
  "backend/app/config.py": """from pydantic import BaseModel
from pydantic_settings import BaseSettings, SettingsConfigDict
import os


class Settings(BaseSettings):
    APP_ENV: str = "dev"
    DATA_DIR: str = "./data"
    UPLOAD_DIR: str = "./uploads"
    INDEX_DIR: str = "./indexes"
    DATABASE_URL: str = "sqlite:///./data/app.db"

    EMBEDDINGS_PROVIDER: str = "huggingface"  # huggingface | openai
    EMBEDDINGS_MODEL: str = "sentence-transformers/all-MiniLM-L6-v2"

    OPENAI_API_KEY: str | None = None
    LLM_MODEL: str = "gpt-4o-mini"

    MAX_CHUNK_TOKENS: int = 600
    CHUNK_OVERLAP_TOKENS: int = 80

    LOG_LEVEL: str = "INFO"

    model_config = SettingsConfigDict(env_file=os.getenv("ENV_FILE", ".env"),
env_file_encoding="utf-8")
```

```
settings = Settings()

# Ensure dirs exist
for d in (settings.DATA_DIR, settings.UPLOAD_DIR, settings.INDEX_DIR):
    os.makedirs(d, exist_ok=True)
""",
  "backend/app/logging_config.py": """import logging
import sys
from .config import settings

def configure_logging():
    level = getattr(logging, settings.LOG_LEVEL.upper(), logging.INFO)
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)s [%(name)s] %(message)s",
        handlers=[logging.StreamHandler(sys.stdout)],
    )
""",
  "backend/app/utils/__init__.py": """""",
  "backend/app/utils/file_utils.py": """import hashlib
import os
from datetime import datetime
from typing import Tuple

def safe_filename(name: str) -> str:
    keep = [c for c in name if c.isalnum() or c in (' ', '.', '_', '-',)]
    s = ''.join(keep).strip().replace(' ', '_')
    return s or datetime.utcnow().strftime('%Y%m%d%H%M%S')

def hash_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def store_upload(base_dir: str, file_bytes: bytes, original_name: str) -> Tuple[str, str]:
    os.makedirs(base_dir, exist_ok=True)
    digest = hash_bytes(file_bytes)[:16]
    clean = safe_filename(original_name)
    fname = f"{digest}__{clean}"
    path = os.path.join(base_dir, fname)
    with open(path, "wb") as f:
        f.write(file_bytes)
    return fname, path
""",
  "backend/app/models/__init__.py": """""",
  "backend/app/models/db_models.py": """from typing import Optional
from sqlmodel import SQLModel, Field
from datetime import datetime


class Book(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    title: str
    filename: str
    filepath: str
    status: str = "PENDING"  # PENDING | INDEXING | READY | ERROR
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)


class Chunk(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    book_id: int = Field(index=True)
```

```
    topic: str = Field(index=True)
    subtopic: str | None = Field(default=None, index=True)
    difficulty: str = Field(default="medium", index=True)  # easy|medium|hard
    start_page: int = 0
    end_page: int = 0
    token_count: int = 0
    text_preview: str = ""  # small preview for UI
""",
  "backend/app/models/schemas.py": """from pydantic import BaseModel, Field
from typing import List, Optional


class BookCreate(BaseModel):
    title: str


class BookRead(BaseModel):
    id: int
    title: str
    filename: str
    status: str

    class Config:
        from_attributes = True


class Topic(BaseModel):
    topic: str
    subtopics: List[str] = Field(default_factory=list)


class UploadResponse(BaseModel):
    book: BookRead
    message: str


class ExamRequest(BaseModel):
    book_id: int
    topics: List[str]
    difficulty: str = "medium"
    num_questions: int = 10


class Question(BaseModel):
    id: str
    topic: str
    prompt: str
    options: Optional[List[str]] = None
    answer: Optional[str] = None
    explanation: Optional[str] = None
    difficulty: str = "medium"


class ExamResponse(BaseModel):
    title: str
    questions: List[Question]
""",
  "backend/app/db/__init__.py": """""",
  "backend/app/db/session.py": """from sqlmodel import create_engine, SQLModel, Session
from ..config import settings
import os
```

```
connect_args = {}
if settings.DATABASE_URL.startswith("sqlite"):
    connect_args = {"check_same_thread": False}

engine = create_engine(settings.DATABASE_URL, echo=False, connect_args=connect_args)

def init_db():
    from ..models.db_models import Book, Chunk  # noqa
    SQLModel.metadata.create_all(engine)

def get_session():
    return Session(engine)
""",
  "backend/app/db/vectorstore.py": """from typing import Optional, List
import os
from langchain_community.vectorstores import FAISS
from langchain.docstore.document import Document
from ..processing.embedding import get_embeddings
from ..config import settings
import logging

logger = logging.getLogger(__name__)


def book_index_dir(book_id: int) -> str:
    d = os.path.join(settings.INDEX_DIR, f"book_{book_id}")
    os.makedirs(d, exist_ok=True)
    return d

def save_faiss(store: FAISS, book_id: int):
    path = book_index_dir(book_id)
    store.save_local(path)
    logger.info(f"Saved FAISS index for book {book_id} to {path}")

def load_faiss(book_id: int) -> Optional[FAISS]:
    path = book_index_dir(book_id)
    if not os.path.exists(path):
        return None
    try:
        embeddings = get_embeddings()
        store = FAISS.load_local(path, embeddings, allow_dangerous_deserialization=True)
        return store
    except Exception as e:
        logger.exception(f"Failed to load FAISS for book {book_id}: {e}")
        return None

def upsert_documents(book_id: int, docs: List[Document]) -> FAISS:
    store = load_faiss(book_id)
    embeddings = get_embeddings()
    if store is None:
        store = FAISS.from_documents(docs, embeddings)
    else:
        store.add_documents(docs)
    save_faiss(store, book_id)
    return store
""",
  "backend/app/processing/__init__.py": """""",
  "backend/app/processing/parser.py": """from typing import List, Dict, Any
import fitz  # PyMuPDF
import re
```

```
import logging

logger = logging.getLogger(__name__)

HEADING_RE =
re.compile(r'^(Cap[ií]tulo|Tema|Unidad|Secci[oó]n)\\s+\\d+\\b|^[A-ZÁÉÍÓÚÑ][\\w\\s]{0,60}$'
EXERCISE_RE = re.compile(r'^(Ejercicio|Problema|Actividad)\\s*\\d*\\b', re.IGNORECASE)

class ParsedSection:
    def __init__(self, topic: str, subtopic: str | None, text: str, pages: tuple[int, int]
str = "content"):
        self.topic = topic
        self.subtopic = subtopic
        self.text = text
        self.pages = pages
        self.kind = kind  # content | exercise


def parse_pdf(path: str) -> List[ParsedSection]:
    doc = fitz.open(path)
    sections: List[ParsedSection] = []
    current_topic = "General"
    current_subtopic = None
    buffer: list[str] = []
    start_page = 0
    def flush(kind="content"):
        nonlocal buffer, start_page, current_topic, current_subtopic
        if buffer:
            text = "\\n".join(buffer).strip()
            if text:
                sections.append(ParsedSection(current_topic, current_subtopic, text, (star
page_no), kind))
        buffer = []

    for page_no in range(len(doc)):
        page = doc.load_page(page_no)
        text = page.get_text("text")
        lines = [l.strip() for l in text.splitlines() if l.strip()]
        for line in lines:
            # Detect exercise blocks
            if EXERCISE_RE.match(line):
                flush(kind="content")
                start_page = page_no
                current_subtopic_local = (current_subtopic or current_topic) + " - Ejercic
                sections.append(ParsedSection(current_topic, current_subtopic_local, line,
(page_no, page_no), "exercise"))
                continue
            # Detect headings
            if HEADING_RE.match(line) and len(line.split()) <= 12:
                flush(kind="content")
                start_page = page_no
                # Heuristic: if all uppercase or short, treat as topic/subtopic
                if current_topic == "General":
                    current_topic = line
                    current_subtopic = None
                else:
                    current_subtopic = line
                continue
            # Accumulate text
            if not buffer:
                start_page = page_no
```

```
            buffer.append(line)
        # flush small exercise buffers per page end? Keep accumulating
    flush(kind="content")
    logger.info(f"Parsed {len(sections)} sections from PDF {path}")
    return sections


def parse_file(path: str) -> List[ParsedSection]:
    if path.lower().endswith(".pdf"):
        return parse_pdf(path)
    # Future: add EPUB, DOCX parsing
    raise ValueError(f"Formato no soportado por ahora: {path}")
""",
  "backend/app/processing/chunker.py": """from typing import List, Dict, Any, Tuple
import tiktoken
from .parser import ParsedSection
from ..config import settings
import logging

logger = logging.getLogger(__name__)

def estimate_difficulty(text: str) -> str:
    # Simple heuristic: more formulas and longer sentences => harder
    score = 0
    score += text.count("=") + text.count("\\int") + text.count("\\sum")
    score += text.count("demuest") + text.count("pruebe")
    words = len(text.split())
    if score >= 5 or words > 400:
        return "hard"
    if score >= 2 or words > 200:
        return "medium"
    return "easy"

def tokenize_len(text: str, model: str = "cl100k_base") -> int:
    try:
        enc = tiktoken.get_encoding(model)
    except Exception:
        enc = tiktoken.get_encoding("cl100k_base")
    return len(enc.encode(text))

def chunk_section(section: ParsedSection, max_tokens: int, overlap: int) -> List[Dict[str,
    text = section.text
    # naive sentence split
    sentences = [s.strip() for s in text.replace("\\n", " ").split(". ") if s.strip()]
    chunks = []
    curr = ""
    for s in sentences:
        candidate = (curr + " " + s).strip()
        if tokenize_len(candidate) > max_tokens:
            if curr:
                chunks.append(curr.strip())
            curr = s
        else:
            curr = candidate
    if curr:
        chunks.append(curr.strip())
    # apply overlap by merging previous tail
    final_chunks: List[str] = []
    prev = ""
    for i, c in enumerate(chunks):
        if prev and overlap > 0:
```

```
            tail_words = prev.split()[-min(len(prev.split()), 50):]
            c = " ".join(tail_words) + " " + c
        final_chunks.append(c)
        prev = c
    results = []
    for c in final_chunks:
        results.append({
            "topic": section.topic,
            "subtopic": section.subtopic,
            "difficulty": estimate_difficulty(c) if section.kind != "exercise" else "mediu
            "text": c,
            "pages": section.pages,
            "kind": section.kind
        })
    return results

def build_chunks(sections: List[ParsedSection]) -> List[Dict[str, Any]]:
    mx = settings.MAX_CHUNK_TOKENS
    ov = settings.CHUNK_OVERLAP_TOKENS
    out: List[Dict[str, Any]] = []
    for sec in sections:
        out.extend(chunk_section(sec, mx, ov))
    logger.info(f"Built {len(out)} chunks from {len(sections)} sections")
    return out
""",
  "backend/app/processing/embedding.py": """from langchain_community.embeddings import
HuggingFaceEmbeddings
from langchain_openai import OpenAIEmbeddings
from ..config import settings
import os

_embeddings_cache = None

def get_embeddings():
    global _embeddings_cache
    if _embeddings_cache is not None:
        return _embeddings_cache
    if settings.EMBEDDINGS_PROVIDER.lower() == "openai" and settings.OPENAI_API_KEY:
        _embeddings_cache = OpenAIEmbeddings(model="text-embedding-3-small",
api_key=settings.OPENAI_API_KEY)
    else:
        _embeddings_cache = HuggingFaceEmbeddings(model_name=settings.EMBEDDINGS_MODEL)
    return _embeddings_cache
""",
  "backend/app/retrieval/__init__.py": """""",
  "backend/app/retrieval/retriever.py": """from typing import List, Dict, Any
from langchain.docstore.document import Document
from ..db.vectorstore import load_faiss
from sqlmodel import select
from ..db.session import get_session
from ..models.db_models import Chunk

def get_topics(book_id: int) -> List[str]:
    with get_session() as s:
        res = s.exec(select(Chunk.topic).where(Chunk.book_id == book_id)).all()
    return sorted(list({r for r in res if r}))

def retrieve_by_topics(book_id: int, topics: List[str], k: int = 4) -> List[Document]:
    store = load_faiss(book_id)
    if not store:
        return []
```

```
    # Filter metadata by topic
    results = []
    for t in topics:
        docs = store.similarity_search(query=t, k=k, filter={"topic": t})
        results.extend(docs)
    return results
""",
  "backend/app/services/__init__.py": """""",
  "backend/app/services/indexer.py": """from typing import List
from langchain.docstore.document import Document
from ..processing.parser import parse_file
from ..processing.chunker import build_chunks
from ..db.vectorstore import upsert_documents
from ..db.session import get_session
from ..models.db_models import Book, Chunk
import logging
from datetime import datetime

logger = logging.getLogger(__name__)

def index_book(book: Book):
    logger.info(f"Indexing book {book.id} - {book.title}")
    with get_session() as s:
        book.status = "INDEXING"
        book.updated_at = datetime.utcnow()
        s.add(book)
        s.commit()
        s.refresh(book)
    try:
        sections = parse_file(book.filepath)
        chunks = build_chunks(sections)
        docs: List[Document] = []
        db_chunks: List[Chunk] = []
        for ch in chunks:
            meta = {
                "book_id": book.id,
                "topic": ch["topic"],
                "subtopic": ch.get("subtopic"),
                "difficulty": ch.get("difficulty"),
                "pages": ch.get("pages"),
                "kind": ch.get("kind"),
                "title": book.title,
            }
            docs.append(Document(page_content=ch["text"], metadata=meta))
            db_chunks.append(Chunk(
                book_id=book.id,
                topic=ch["topic"],
                subtopic=ch.get("subtopic"),
                difficulty=ch.get("difficulty", "medium"),
                start_page=ch.get("pages", (0,0))[0],
                end_page=ch.get("pages", (0,0))[1],
                token_count=len(ch["text"].split()),
                text_preview=ch["text"][:200]
            ))
        upsert_documents(book.id, docs)
        with get_session() as s:
            # Clear existing metadata for idempotency
            s.exec(f"DELETE FROM chunk WHERE book_id = {book.id}")
            for c in db_chunks:
                s.add(c)
            book.status = "READY"
```

```
            book.updated_at = datetime.utcnow()
            s.add(book)
            s.commit()
        logger.info(f"Indexed {len(docs)} chunks for book {book.id}")
    except Exception as e:
        logger.exception(f"Error indexing book {book.id}: {e}")
        with get_session() as s:
            book.status = "ERROR"
            book.updated_at = datetime.utcnow()
            s.add(book)
            s.commit()
        raise
""",
  "backend/app/services/exam_generator.py": """from typing import List
import uuid
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from ..config import settings
from ..retrieval.retriever import retrieve_by_topics
from ..models.schemas import ExamResponse, Question
import logging

logger = logging.getLogger(__name__)

FALLBACK_MC_OPTIONS = ["A", "B", "C", "D"]

def use_openai() -> bool:
    return bool(settings.OPENAI_API_KEY and settings.LLM_MODEL)

def build_prompt(context: str, topic: str, difficulty: str, num_q: int) -> str:
    template = """Eres un generador de exámenes para estudiantes.
Crea {num_q} preguntas de opción múltiple de dificultad {difficulty} sobre el tema: "{topi
Usa únicamente el siguiente contexto del libro, preserva notación y precisión.
Proporciona JSON con formato:
[
  {{"id": "uuid", "topic": "string", "prompt": "string", "options": ["A) ...","B) ...","C)
..."], "answer": "A|B|C|D", "explanation": "string", "difficulty":"{difficulty}"}},
  ...
]
Contexto:
{context}
"""
    return PromptTemplate.from_template(template).format(topic=topic, difficulty=difficult
context=context[:8000], num_q=num_q)

def generate_exam(book_id: int, topics: List[str], difficulty: str, num_questions: int) ->
ExamResponse:
    per_topic = max(1, num_questions // max(1, len(topics)))
    questions: List[Question] = []
    for t in topics:
        docs = retrieve_by_topics(book_id, [t], k=6)
        context = "\\n\\n".join([d.page_content for d in docs])
        if use_openai():
            try:
                llm = ChatOpenAI(model=settings.LLM_MODEL, api_key=settings.OPENAI_API_KEY
temperature=0.3)
                prompt = build_prompt(context, t, difficulty, per_topic)
                resp = llm.invoke(prompt)
                import json
                data = json.loads(resp.content)
                for q in data[:per_topic]:
```

```
                    questions.append(Question(
                        id=q.get("id") or str(uuid.uuid4()),
                        topic=q.get("topic") or t,
                        prompt=q.get("prompt") or "",
                        options=q.get("options"),
                        answer=q.get("answer"),
                        explanation=q.get("explanation"),
                        difficulty=q.get("difficulty") or difficulty
                    ))
                continue
            except Exception as e:
                logger.warning(f"Fallo LLM, usando fallback: {e}")
        # Fallback: simple generation from context
        lines = [ln.strip() for ln in context.splitlines() if len(ln.strip()) > 20]
        for i in range(min(per_topic, len(lines))):
            base = lines[i][:180].rstrip(".")
            prompt = f"{base} ¿Cuál es la opción correcta?"
            options = [f"{opt}) {base}." for opt in ["A", "B", "C", "D"]]
            answer = "A"
            questions.append(Question(
                id=str(uuid.uuid4()),
                topic=t,
                prompt=prompt,
                options=options,
                answer=answer,
                explanation="Basado en el contexto proporcionado.",
                difficulty=difficulty
            ))
    # If still not enough, pad with generic
    while len(questions) < num_questions:
        questions.append(Question(
            id=str(uuid.uuid4()),
            topic=topics[0] if topics else "General",
            prompt="Pregunta de ejemplo: seleccione la opción correcta.",
            options=[f"{o}) Opción {o}" for o in ["A", "B", "C", "D"]],
            answer="A",
            explanation="Ejemplo.",
            difficulty=difficulty
        ))
    return ExamResponse(title="Examen Personalizado", questions=questions[:num_questions])
""",
  "backend/app/main.py": """from fastapi import FastAPI, UploadFile, File, Form, Backgroun
HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from .config import settings
from .logging_config import configure_logging
from .db.session import init_db, get_session
from .models.db_models import Book, Chunk
from .models.schemas import BookRead, UploadResponse, ExamRequest, ExamResponse, Topic
from .utils.file_utils import store_upload
from .services.indexer import index_book
from .services.exam_generator import generate_exam
from .retrieval.retriever import get_topics
from sqlmodel import select
from datetime import datetime
import logging
import os

configure_logging()
logger = logging.getLogger(__name__)
```

```
app = FastAPI(title="LLM Exam Builder", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
def on_startup():
    init_db()
    logger.info("API started")

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/api/books/upload", response_model=UploadResponse)
async def upload_book(background_tasks: BackgroundTasks, file: UploadFile = File(...), tit
Form(...)):
    try:
        if not file.filename.lower().endswith(".pdf"):
            raise HTTPException(400, "Actualmente solo se soportan PDFs")
        data = await file.read()
        fname, path = store_upload(settings.UPLOAD_DIR, data, file.filename)
        book = Book(title=title, filename=fname, filepath=path, status="PENDING")
        with get_session() as s:
            s.add(book)
            s.commit()
            s.refresh(book)
        background_tasks.add_task(index_book, book)
        return UploadResponse(book=BookRead.model_validate(book), message="Subido y en pro
indexación")
    except Exception as e:
        logger.exception("Error al subir libro")
        raise HTTPException(500, f"Error al subir libro: {e}")

@app.get("/api/books", response_model=list[BookRead])
def list_books():
    with get_session() as s:
        books = s.exec(select(Book).order_by(Book.created_at.desc())).all()
    return [BookRead.model_validate(b) for b in books]

@app.get("/api/books/{book_id}/topics", response_model=list[Topic])
def list_topics(book_id: int):
    topics = get_topics(book_id)
    return [{"topic": t, "subtopics": []} for t in topics]

@app.post("/api/index/reindex/{book_id}")
def reindex_book(book_id: int, background_tasks: BackgroundTasks):
    with get_session() as s:
        book = s.get(Book, book_id)
        if not book:
            raise HTTPException(404, "Libro no encontrado")
        book.status = "PENDING"
        book.updated_at = datetime.utcnow()
        s.add(book)
        s.commit()
```

```
    background_tasks.add_task(index_book, book)
    return {"message": "Reindexación iniciada"}

@app.post("/api/exams/generate", response_model=ExamResponse)
def api_generate_exam(req: ExamRequest):
    # validate
    if req.num_questions <= 0:
        raise HTTPException(400, "num_questions debe ser > 0")
    resp = generate_exam(req.book_id, req.topics, req.difficulty, req.num_questions)
    return JSONResponse(content=resp.model_dump())
""",
  "backend/tests/test_health.py": """def test_placeholder():
    assert True
""",
  "frontend/package.json": """{
  "name": "exam-builder-frontend",
  "version": "0.1.0",
  "private": true,
  "main": "app/App.tsx",
  "scripts": {
    "start": "expo start --web",
    "android": "expo run:android",
    "ios": "expo run:ios",
    "web": "expo start --web",
    "lint": "echo \"No lint configured\"",
    "tsc": "tsc --noEmit"
  },
  "dependencies": {
    "expo": "~51.0.28",
    "expo-constants": "~16.0.2",
    "expo-status-bar": "~1.12.1",
    "react": "18.2.0",
    "react-dom": "18.2.0",
    "react-native": "0.74.5",
    "react-native-safe-area-context": "4.10.5",
    "react-native-web": "~0.19.13",
    "@react-navigation/native": "^6.1.17",
    "@react-navigation/native-stack": "^6.10.0"
  },
  "devDependencies": {
    "@types/react": "~18.2.79",
    "@types/react-native": "~0.73.0",
    "typescript": "^5.6.2"
  }
}
""",
  "frontend/app.json": """{
  "expo": {
    "name": "Exam Builder",
    "slug": "exam-builder",
    "scheme": "exam-builder",
    "version": "0.1.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "automatic",
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "web": {
```

```
      "bundler": "metro",
      "favicon": "./assets/favicon.png"
    }
  }
}
""",
  "frontend/babel.config.js": """module.exports = function(api) {
  api.cache(true);
  return {
    presets: ['babel-preset-expo']
  };
};
""",
  "frontend/metro.config.js": """const { getDefaultConfig } = require('expo/metro-config')

const config = getDefaultConfig(__dirname);

module.exports = config;
""",
  "frontend/tsconfig.json": """{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true
  }
}
""",
  "frontend/app/api/client.ts": """const BASE_URL = process.env.API_BASE_URL ||
'http://localhost:8000';

export async function uploadBook(file: File, title: string) {
  const form = new FormData();
  form.append('file', file);
  form.append('title', title);
  const res = await fetch(`${BASE_URL}/api/books/upload`, {
    method: 'POST',
    body: form
  });
  if (!res.ok) throw new Error(await res.text());
  return await res.json();
}

export async function getBooks() {
  const res = await fetch(`${BASE_URL}/api/books`);
  if (!res.ok) throw new Error(await res.text());
  return await res.json();
}

export async function getTopics(bookId: number) {
  const res = await fetch(`${BASE_URL}/api/books/${bookId}/topics`);
  if (!res.ok) throw new Error(await res.text());
  return await res.json();
}

export async function generateExam(body: { book_id: number; topics: string[]; difficulty:
num_questions: number; }) {
  const res = await fetch(`${BASE_URL}/api/exams/generate`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body)
  });
  if (!res.ok) throw new Error(await res.text());
```

```
  return await res.json();
}
""",
  "frontend/app/components/BookItem.tsx": """import React from 'react';

type Props = {
  title: string;
  status: string;
  onPress: () => void;
};

export default function BookItem({ title, status, onPress }: Props) {
  return (
    <div onClick={onPress} style={{
      border: '1px solid #eee', padding: 12, borderRadius: 8, marginBottom: 8, cursor: 'po
    }}>
      <div style={{ fontWeight: 600 }}>{title}</div>
      <div style={{ fontSize: 12, color: '#666' }}>Estado: {status}</div>
    </div>
  );
}
""",
  "frontend/app/components/TopicList.tsx": """import React from 'react';

type Props = {
  topics: string[];
  selected: string[];
  toggle: (t: string) => void;
};

export default function TopicList({ topics, selected, toggle }: Props) {
  return (
    <div style={{ display: 'flex', flexDirection: 'column', gap: 6 }}>
      {topics.map((t) => (
        <label key={t} style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
          <input type="checkbox" checked={selected.includes(t)} onChange={() => toggle(t)}
          <span>{t}</span>
        </label>
      ))}
    </div>
  );
}
""",
  "frontend/app/hooks/useBooks.ts": """import { useEffect, useState } from 'react';
import { getBooks } from '../api/client';

export function useBooks() {
  const [books, setBooks] = useState<any[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function refresh() {
    setLoading(true);
    setError(null);
    try {
      const data = await getBooks();
      setBooks(data);
    } catch (e: any) {
      setError(e.message || 'Error');
    } finally {
      setLoading(false);
```

```
    }
  }

  useEffect(() => {
    refresh();
  }, []);

  return { books, loading, error, refresh };
}
""",
  "frontend/app/screens/LibraryScreen.tsx": """import React from 'react';
import { useBooks } from '../hooks/useBooks';
import BookItem from '../components/BookItem';

type Props = {
  goToUpload: () => void;
  openBook: (book: any) => void;
};

export default function LibraryScreen({ goToUpload, openBook }: Props) {
  const { books, loading, error, refresh } = useBooks();

  return (
    <div style={{ padding: 16, maxWidth: 900, margin: '0 auto' }}>
      <h2>Biblioteca</h2>
      <button onClick={goToUpload} style={{ marginBottom: 12 }}>Subir libro</button>
      <button onClick={refresh} style={{ marginLeft: 8 }}>Refrescar</button>
      {loading && <div>Cargando...</div>}
      {error && <div style={{ color: 'red' }}>{error}</div>}
      <div>
        {books.map((b) => (
          <BookItem key={b.id} title={b.title} status={b.status} onPress={() => openBook(b
        ))}
      </div>
    </div>
  );
}
""",
  "frontend/app/screens/UploadScreen.tsx": """import React, { useState, useRef } from 'rea
import { uploadBook } from '../api/client';

type Props = {
  goBack: () => void;
};

export default function UploadScreen({ goBack }: Props) {
  const [title, setTitle] = useState('');
  const [busy, setBusy] = useState(false);
  const [msg, setMsg] = useState<string | null>(null);
  const fileRef = useRef<HTMLInputElement>(null);

  async function onSubmit(e: React.FormEvent) {
    e.preventDefault();
    const file = fileRef.current?.files?.[0];
    if (!file || !title) {
      setMsg('Seleccione un PDF y escriba un título');
      return;
    }
    setBusy(true);
    setMsg(null);
    try {
```

```
      const res = await uploadBook(file, title);
      setMsg(`Subido: ${res.book.title}. Estado: ${res.book.status}`);
    } catch (e: any) {
      setMsg(e.message || 'Error al subir');
    } finally {
      setBusy(false);
    }
  }

  return (
    <div style={{ padding: 16, maxWidth: 600, margin: '0 auto' }}>
      <h2>Subir nuevo libro (PDF)</h2>
      <form onSubmit={onSubmit} style={{ display: 'flex', flexDirection: 'column', gap: 12
        <label>Título
          <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Álg
Lineal - Ejercicios" />
        </label>
        <label>Archivo PDF
          <input type="file" accept="application/pdf" ref={fileRef} />
        </label>
        <div style={{ display: 'flex', gap: 8 }}>
          <button type="submit" disabled={busy}>{busy ? 'Subiendo...' : 'Subir'}</button>
          <button type="button" onClick={goBack}>Volver</button>
        </div>
      </form>
      {msg && <div style={{ marginTop: 12 }}>{msg}</div>}
    </div>
  );
}
""",
  "frontend/app/screens/BookScreen.tsx": """import React, { useEffect, useState } from 're
import { getTopics } from '../api/client';
import TopicList from '../components/TopicList';

type Props = {
  book: any;
  goBack: () => void;
  goToExamConfig: (selectedTopics: string[]) => void;
};

export default function BookScreen({ book, goBack, goToExamConfig }: Props) {
  const [topics, setTopics] = useState<string[]>([]);
  const [selected, setSelected] = useState<string[]>([]);
  const [loading, setLoading] = useState(false);
  const [err, setErr] = useState<string | null>(null);

  useEffect(() => {
    async function load() {
      setLoading(true);
      setErr(null);
      try {
        const res = await getTopics(book.id);
        setTopics(res.map((t: any) => t.topic));
      } catch (e: any) {
        setErr(e.message || 'Error');
      } finally {
        setLoading(false);
      }
    }
    load();
  }, [book.id]);
```

```
  function toggle(t: string) {
    setSelected((prev) => prev.includes(t) ? prev.filter(x => x !== t) : [...prev, t]);
  }

  return (
    <div style={{ padding: 16, maxWidth: 900, margin: '0 auto' }}>
      <h2>{book.title}</h2>
      <div>Estado: <b>{book.status}</b></div>
      {loading && <div>Cargando temas...</div>}
      {err && <div style={{ color: 'red' }}>{err}</div>}
      {!loading && topics.length === 0 && <div>No hay temas aún. Si el libro fue subido
recientemente, espere a que termine la indexación.</div>}
      <div style={{ marginTop: 12 }}>
        <TopicList topics={topics} selected={selected} toggle={toggle} />
      </div>
      <div style={{ marginTop: 12, display: 'flex', gap: 8 }}>
        <button onClick={goBack}>Volver</button>
        <button onClick={() => goToExamConfig(selected)} disabled={selected.length ===
0}>Configurar examen</button>
      </div>
    </div>
  );
}
""",
  "frontend/app/screens/ExamConfigScreen.tsx": """import React, { useState } from 'react';

type Props = {
  topics: string[];
  proceed: (numQuestions: number, difficulty: string) => void;
  goBack: () => void;
};

export default function ExamConfigScreen({ topics, proceed, goBack }: Props) {
  const [num, setNum] = useState(10);
  const [difficulty, setDifficulty] = useState('medium');
  return (
    <div style={{ padding: 16, maxWidth: 600, margin: '0 auto' }}>
      <h2>Configurar examen</h2>
      <div>Temas seleccionados: {topics.join(', ')}</div>
      <div style={{ display: 'flex', flexDirection: 'column', gap: 12, marginTop: 12 }}>
        <label>Cantidad de preguntas
          <input type="number" value={num} onChange={(e) => setNum(parseInt(e.target.value
min={1} />
        </label>
        <label>Dificultad
          <select value={difficulty} onChange={(e) => setDifficulty(e.target.value)}>
            <option value="easy">Fácil</option>
            <option value="medium">Media</option>
            <option value="hard">Difícil</option>
          </select>
        </label>
      </div>
      <div style={{ marginTop: 12, display: 'flex', gap: 8 }}>
        <button onClick={goBack}>Volver</button>
        <button onClick={() => proceed(num, difficulty)} disabled={num <= 0}>Generar</butt
      </div>
    </div>
  );
}
""",
```

```
  "frontend/app/screens/ExamViewScreen.tsx": """import React from 'react';

type Props = {
  exam: any;
  goBack: () => void;
};

export default function ExamViewScreen({ exam, goBack }: Props) {
  return (
    <div style={{ padding: 16, maxWidth: 900, margin: '0 auto' }}>
      <h2>{exam?.title || 'Examen'}</h2>
      <div style={{ display: 'flex', flexDirection: 'column', gap: 16 }}>
        {exam?.questions?.map((q: any, idx: number) => (
          <div key={q.id} style={{ border: '1px solid #eee', padding: 12, borderRadius: 8
            <div style={{ fontWeight: 600 }}>{idx + 1}. [{q.topic}] ({q.difficulty})</div>
            <div style={{ marginTop: 8 }}>{q.prompt}</div>
            {q.options && (
              <ul>
                {q.options.map((opt: string) => <li key={opt}>{opt}</li>)}
              </ul>
            )}
            {q.answer && <div><b>Respuesta:</b> {q.answer}</div>}
            {q.explanation && <div><b>Explicación:</b> {q.explanation}</div>}
          </div>
        ))}
      </div>
      <div style={{ marginTop: 12 }}>
        <button onClick={goBack}>Volver</button>
      </div>
    </div>
  );
}
""",
  "frontend/app/App.tsx": """import React, { useState } from 'react';
import LibraryScreen from './screens/LibraryScreen';
import UploadScreen from './screens/UploadScreen';
import BookScreen from './screens/BookScreen';
import ExamConfigScreen from './screens/ExamConfigScreen';
import ExamViewScreen from './screens/ExamViewScreen';
import { generateExam } from './api/client';

type View =
  | { name: 'library' }
  | { name: 'upload' }
  | { name: 'book', book: any }
  | { name: 'exam-config', book: any, topics: string[] }
  | { name: 'exam-view', exam: any };

export default function App() {
  const [view, setView] = useState<View>({ name: 'library' });

  if (view.name === 'library') {
    return <LibraryScreen goToUpload={() => setView({ name: 'upload' })} openBook={(book)
setView({ name: 'book', book })} />;
  }

  if (view.name === 'upload') {
    return <UploadScreen goBack={() => setView({ name: 'library' })} />;
  }

  if (view.name === 'book') {
```

```
    return <BookScreen
      book={view.book}
      goBack={() => setView({ name: 'library' })}
      goToExamConfig={(topics) => setView({ name: 'exam-config', book: view.book, topics }
    />;
  }

  if (view.name === 'exam-config') {
    const proceed = async (num: number, difficulty: string) => {
      const exam = await generateExam({ book_id: view.book.id, topics: view.topics, diffic
num_questions: num });
      setView({ name: 'exam-view', exam });
    };
    return <ExamConfigScreen topics={view.topics} proceed={proceed} goBack={() => setView(
'book', book: view.book })} />;
  }

  if (view.name === 'exam-view') {
    return <ExamViewScreen exam={view.exam} goBack={() => setView({ name: 'library' })} />
  }

  return <div>Pantalla no encontrada</div>;
}
"""
}
```